

IA como sistema

Funções, processos, contratos e validação

Até aqui, a disciplina mostrou a inteligência artificial em camadas diferentes. Nas primeiras aulas, o foco esteve no ambiente profissional de IA: versionamento, experimentos, métricas, pipelines e MLOps. Depois, o olhar foi para a IA generativa discutida como função e como processo. E agora, vamos ao próximo passo: pensar IA como sistema.

Essa mudança parece pequena, mas altera a forma de projetar soluções. Quando usamos IA como função, a estrutura básica é simples: uma entrada, um prompt e uma resposta. Quando usamos IA como processo, organizamos uma sequência de ações: primeiro interpretar, depois gerar, depois revisar, por exemplo. Ao tratar IA como sistema, acrescentamos um elemento decisivo: cada parte do fluxo passa a ter responsabilidade definida, entrada esperada, saída esperada e algum tipo de validação.

Em outras palavras, a perspectiva passa a ser a organização de componentes para que a solução funcione de modo controlado, testável e reaproveitável.

O problema do prompt único

Um erro comum em IA generativa é tentar resolver tudo com um único prompt. O usuário descreve uma tarefa grande, mistura contexto, regras, exemplos, formato de saída, restrições e critérios de qualidade, e espera que o modelo resolva tudo em uma única resposta. Esse método pode funcionar em situações simples, mas costuma falhar quando a tarefa exige consistência.

Um pedido típico com excesso de responsabilidades pode aparecer assim:

Analise a dúvida do aluno, identifique o tema, escolha a estratégia didática, responda de forma clara, verifique se há erro conceitual, adapte a linguagem ao curso e devolva tudo em JSON.

Há muitas tarefas dentro de uma só instrução. O modelo precisa interpretar, classificar, decidir, gerar, revisar e formatar ao mesmo tempo. Quando algo sai errado, é difícil descobrir onde ocorreu a falha. A origem do problema pode estar na interpretação da dúvida, na escolha da estratégia, na resposta gerada, no formato JSON ou na ausência de validação.

Ao transformar esse uso em sistema, dividimos o problema em partes menores. Cada parte fica mais fácil de entender, testar e melhorar.

A arquitetura mínima de um sistema de IA generativa

Um sistema simples de IA generativa pode ser organizado em quatro blocos:

Entrada -> Processamento -> Validação -> Saída

O valor desta abordagem está na separação de responsabilidades. Isso está muito além de apenas indicar uma ordem.

A entrada reúne os dados que iniciam o fluxo. Pode incluir a solicitação do usuário, documentos de apoio, histórico, parâmetros, restrições de formato ou dados recuperados de outra fonte. Em um sistema bem projetado, a entrada não é tratada como um texto solto. Ela precisa ser organizada para que o restante do fluxo saiba exatamente o que recebeu.

O processamento transforma a entrada em algum resultado intermediário ou final. Essa etapa pode usar um modelo de linguagem, um prompt estruturado, uma função em Python, uma consulta a banco de dados, uma ferramenta externa ou uma combinação dessas opções. O ponto importante é evitar que uma única etapa concentre responsabilidades demais.

A validação verifica se o resultado atende a critérios definidos. Ela pode conferir formato, campos obrigatórios, valores permitidos, consistência com o contexto, aderência a regras de negócio ou presença de informações mínimas. Validação não é detalhe: é o que separa um uso experimental de um sistema mais confiável.

A saída organiza o resultado para quem vai consumi-lo. Esse consumidor pode ser uma pessoa, outro sistema, uma API, um relatório, um arquivo JSON, uma tabela ou uma etapa seguinte do próprio pipeline. Por isso, uma saída "boa" não é apenas uma resposta bem escrita. Ela precisa estar no formato correto e servir ao objetivo do fluxo.

Contratos: o elemento central da aula

Um contrato define exatamente o que uma etapa recebe e o que ela deve entregar. Ele funciona como uma interface entre componentes. Sem contrato, cada etapa depende de interpretação livre. Com contrato, o sistema passa a ter uma expectativa verificável.

Considere uma etapa chamada `classificar_tarefa`. Ela poderia receber:

```
{
  "solicitacao": "Explique a diferença entre overfitting e underfitting para um
aluno iniciante."
}
```

E deveria devolver:

```
{
  "tipo": "explicacao_conceitual",
  "publico": "iniciante",
  "tema": "overfitting e underfitting",
  "risco": "confundir erro de treino com erro de generalização"
}
```

Esse contrato deixa claro o papel da etapa. Ela não precisa escrever a explicação final. Sua responsabilidade é classificar a tarefa e apontar informações úteis para a próxima etapa. A geração da resposta pode ficar com outro componente, que receberia a saída anterior como entrada.

A partir dessa classificação, a etapa seguinte poderia produzir:

```
{
  "resposta": "Overfitting ocorre quando o modelo aprende detalhes excessivos dos
dados de treino...",
  "exemplo": "Um modelo que memoriza imagens específicas em vez de aprender
padrões gerais.",
  "alerta_didatico": "Comparar sempre desempenho em treino e validação."
}
```

A vantagem prática está na possibilidade de investigar cada parte quando a resposta final está ruim. A classificação pode ser conferida separadamente, assim como a identificação do público, o risco didático e a aderência da resposta às informações recebidas. Essa capacidade de diagnóstico é uma das razões para usar contratos.

Exemplo prático: de workflow para sistema

Suponha que o objetivo seja criar um assistente para responder dúvidas de alunos em uma disciplina de IA. Um workflow simples poderia ser:

Receber solicitação -> Gerar resposta -> Entregar ao aluno

Esse fluxo é fácil de montar, mas frágil. Ele não separa interpretação, geração e validação. Uma versão mais sistêmica poderia ser:

```
Receber solicitação
-> Classificar dúvida
-> Gerar resposta didática
-> Validar resposta
-> Formatar saída final
```

Cada etapa teria um contrato. Na etapa de classificação, a entrada é a solicitação do aluno. A saída esperada pode conter tema, nível de dificuldade, tipo de dúvida e risco de erro conceitual. Na geração, a entrada é essa classificação mais a solicitação original. A saída deve conter explicação, exemplo e orientação prática. Na validação, a entrada é a resposta gerada. A saída deve indicar se a resposta está completa, se respeita o nível do aluno e se há algum problema conceitual. Na formatação, a resposta é ajustada ao padrão final.

Um validador simples poderia aplicar critérios objetivos. A resposta deve tratar diretamente da solicitação original, trazer exemplo concreto, usar vocabulário adequado ao nível informado, evitar contradições com o material da disciplina e entregar todos os campos exigidos pelo formato final.

Esses critérios podem ser implementados de várias formas. Alguns podem virar regras programadas, como verificar se campos obrigatórios existem em um JSON. Outros podem usar um segundo prompt de revisão. Em sistemas mais críticos, pode haver também revisão humana.

Por que isso melhora a solução

A decomposição em etapas traz quatro benefícios práticos. O primeiro é a testabilidade. Quando cada etapa tem entrada e saída definidas, é possível testá-la separadamente. O professor ou desenvolvedor pode criar exemplos de solicitações e verificar se a classificação está correta antes de avaliar a resposta final.

O segundo é a substituição de componentes. Uma etapa pode ser melhorada sem reescrever todo o sistema. Se o classificador de dúvidas estiver fraco, ele pode ser ajustado. Se o gerador estiver usando linguagem inadequada, apenas o prompt dessa etapa precisa mudar. Se a formatação final for sempre igual, talvez ela nem precise usar IA; pode ser feita por código.

O terceiro é a rastreabilidade. Ao salvar entradas, saídas intermediárias, versões de prompts e resultados de validação, torna-se possível entender o comportamento do sistema ao longo do tempo. Isso ajuda a corrigir erros, comparar versões e justificar decisões.

O quarto é a confiabilidade. Um sistema com validação explícita reduz a chance de entregar respostas incompletas, fora do formato ou incoerentes com o contexto. Ele não elimina erros, mas cria mecanismos para encontrá-los antes que cheguem ao usuário final.

Como avaliar se um pipeline está bem desenhado

Um pipeline de IA generativa não deve ser avaliado apenas pela resposta final. Ele também deve ser analisado pela qualidade de sua arquitetura. Uma arquitetura bem desenhada separa responsabilidades, define entradas e saídas de cada etapa, inclui pelo menos uma validação explícita, registra informações suficientes para investigar falhas, permite substituir componentes sem reescrever todo o fluxo e entrega uma saída adequada ao usuário ou sistema que vai consumi-la.

Esses critérios forçam uma mudança de postura. O desenvolvedor deixa de pensar apenas como usuário de IA e começa a pensar como projetista de sistemas com IA.

Fechamento

Pensar IA como sistema significa projetar além do modelo. O modelo continua importante, mas passa a operar dentro de uma arquitetura: recebe entradas organizadas, executa uma responsabilidade específica, entrega saídas contratadas e é acompanhado por validações.

Esse é o ponto em que IA generativa se aproxima definitivamente da engenharia de software e de MLOps. O objetivo deixa de ser apenas obter uma boa resposta em uma interação isolada. O objetivo passa a ser construir um fluxo que possa ser testado, mantido, observado e melhorado.

Na prática profissional, essa diferença orienta o projeto técnico. Sistemas frágeis dependem de prompts longos e difíceis de controlar. Sistemas bem desenhados distribuem responsabilidades, registram evidências e validam resultados. A Aula 13 trata exatamente dessa virada: sair do uso pontual da IA e começar a projetar arquiteturas de IA generativa com intenção, controle e valor prático.

Referências

ANTHROPIC. Building Effective Agents. 2024. <https://www.anthropic.com/engineering/building-effective-agents>

HUYEN, Chip. AI Engineering: Building Applications with Foundation Models. O'Reilly, 2025.